

LAB 3. BASIC OBJECT-ORIENTED TECHNIQUES

IT3103E – 162065

Họ và tên: Nguyễn Minh Tuấn

MSSV: 202416759

1. AWTAccumulator

```
1 package hust.soict.dsai.swing;
2
3 import java.awt.*;
4
5
6 public class AWTAccumulator extends Frame {
7     private TextField tfInput;
8     private TextField tfOutput;
9     private int sum = 0; // Accumulated sum, init to 0
10
11     // Constructor to setup the GUI components and event handlers
12 public AWTAccumulator() {
13     setLayout(new GridLayout(2, 2));
14
15     add(new Label("Enter an Integer: "));
16
17     tfInput = new TextField(10);
18     add(tfInput);
19     tfInput.addActionListener(new TFInputListener());
20
21     add(new Label("The Accumulated Sum is: "));
22
23     tfOutput = new TextField(10);
24     tfOutput.setEditable(false);
25     add(tfOutput);
26
27     setTitle("AWT Accumulator");
28     setSize(350, 120);
29 addWindowListener(new WindowAdapter() {
30     @Override
31     public void windowClosing(WindowEvent e) {
32         System.exit(0);
33     }
34 });
35     setVisible(true);
36 }
37
38 public static void main(String[] args) {
39     new AWTAccumulator();
40 }
41
42 private class TFInputListener implements ActionListener {
43     @Override
44     public void actionPerformed(ActionEvent evt) {
45         int numberIn = Integer.parseInt(tfInput.getText());
46         sum += numberIn;
47         tfInput.setText("");
48         tfOutput.setText(sum + "");
49     }
50 }
51 }
52 }
```

AWT Accumulator	
Enter an Integer:	12
The Accumulated Sum is:	

AWT Accumulator	
Enter an Integer:	123
The Accumulated Sum is:	12

AWT Accumulator	
Enter an Integer:	
The Accumulated Sum is:	135

2. SwingAccumulator.java

```
1 package hust.soict.dsai.swing;
2
3 import java.awt.*;
4 import java.awt.event.*;
5 import javax.swing.*;
6
7 public class SwingAccumulator extends JFrame {
8     private JTextField tfInput;
9     private JTextField tfOutput;
10    private int sum = 0; // Accumulated sum, init to 0
11
12    // Constructor to setup the GUI components and event handler
13    public SwingAccumulator() {
14        Container cp = getContentPane();
15        cp.setLayout(new GridLayout(2, 2));
16
17        cp.add(new JLabel("Enter an Integer: "));
18
19        tfInput = new JTextField(10);
20        cp.add(tfInput);
21        tfInput.addActionListener(new TFInputListener());
22
23        cp.add(new JLabel("The Accumulated Sum is: "));
24
25        tfOutput = new JTextField(10);
26        tfOutput.setEditable(false);
27        cp.add(tfOutput);
28
29        setTitle("Swing Accumulator");
30        setSize(350, 120);
31        setVisible(true);
32    }
33
34    public static void main(String[] args) {
35        new SwingAccumulator();
36    }
37
38    private class TFInputListener implements ActionListener {
39        @Override
40        public void actionPerformed(ActionEvent evt) {
41            int numberIn = Integer.parseInt(tfInput.getText());
42            sum += numberIn;
43            tfInput.setText("");
44            tfOutput.setText(sum + "");
45        }
46    }
47 }
```

Swing Accumulator

Enter an Integer:	145
The Accumulated Sum is:	14

Swing Accumulator

Enter an Integer:	14
The Accumulated Sum is:	

Swing Accumulator

Enter an Integer:	
The Accumulated Sum is:	159

3. NumberGrid.java

```

1 package hust.soict.dsai.swing;
2
3 import java.awt.*;
4
5
6
7 public class NumberGrid extends JFrame {
8     private JButton[] btnNumbers = new JButton[10];
9     private JButton btnDelete, btnReset;
10    private JTextField tfDisplay;
11
12    public NumberGrid() {
13        tfDisplay = new JTextField();
14        tfDisplay.setComponentOrientation(
15            ComponentOrientation.RIGHT_TO_LEFT);
16
17        JPanel panelButtons = new JPanel(new GridLayout(4, 3));
18        addButtons(panelButtons); // Phương thức này được gọi nhưng không có trong ảnh
19
20        Container cp = getContentPane();
21        cp.setLayout(new BorderLayout());
22        cp.add(tfDisplay, BorderLayout.NORTH);
23        cp.add(panelButtons, BorderLayout.CENTER);
24
25        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
26        setTitle("Number Grid");
27        setSize(500, 500);
28        setVisible(true);
29    }
30
31    private void addButtons(JPanel panelButtons) {
32        ButtonListener btnListener = new ButtonListener();
33        for (int i = 1; i <= 9; i++) {
34            btnNumbers[i] = new JButton(""+i);
35            panelButtons.add(btnNumbers[i]);
36            btnNumbers[i].addActionListener(btnListener);
37        }
38
39        btnDelete = new JButton("DEL");
40        panelButtons.add(btnDelete);
41        btnDelete.addActionListener(btnListener);
42
43        btnNumbers[0] = new JButton("0");
44        panelButtons.add(btnNumbers[0]);
45        btnNumbers[0].addActionListener(btnListener);
46
47        btnReset = new JButton("C");
48        panelButtons.add(btnReset);
49        btnReset.addActionListener(btnListener);
50    }
51
52    private class ButtonListener implements ActionListener{
53        @Override
54        public void actionPerformed(ActionEvent e) {
55            String button = e.getActionCommand();
56            if (button.charAt(0) >= '0' && button.charAt(0) <= '9') {
57                tfDisplay.setText(tfDisplay.getText() + button);
58            }
59            else if (button.equals("DEL")) {
60                String currentText = tfDisplay.getText();
61                if (currentText.length() > 0) {
62                    String newText = currentText.substring(0, currentText.length() - 1);
63                    tfDisplay.setText(newText);
64                }
65            }
66            else {
67                tfDisplay.setText(null);
68            }
69        }
70    }
71
72    public static void main(String[] args) {
73        new NumberGrid();
74    }
75
76 }

```



4. Painter.java

```
package hust.soict.dsai.javaafx;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class Painter extends Application {

    @Override
    public void start(Stage stage) throws Exception {
        Parent root = FXMLLoader.load(getClass().getResource("/hust/soict/dsai/javaafx/Painter.fxml"));

        Scene scene = new Scene(root);
        stage.setTitle("Painter");
        stage.setScene(scene);
        stage.show();
    }

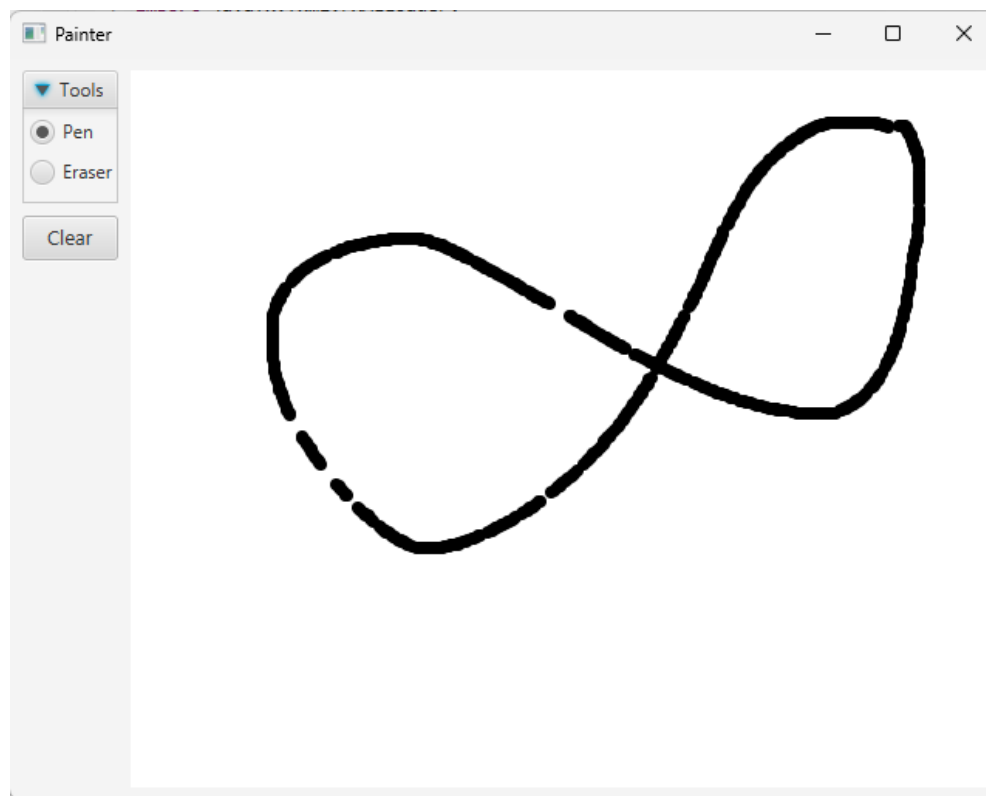
    public static void main(String[] args) {
        launch(args);
    }
}
```

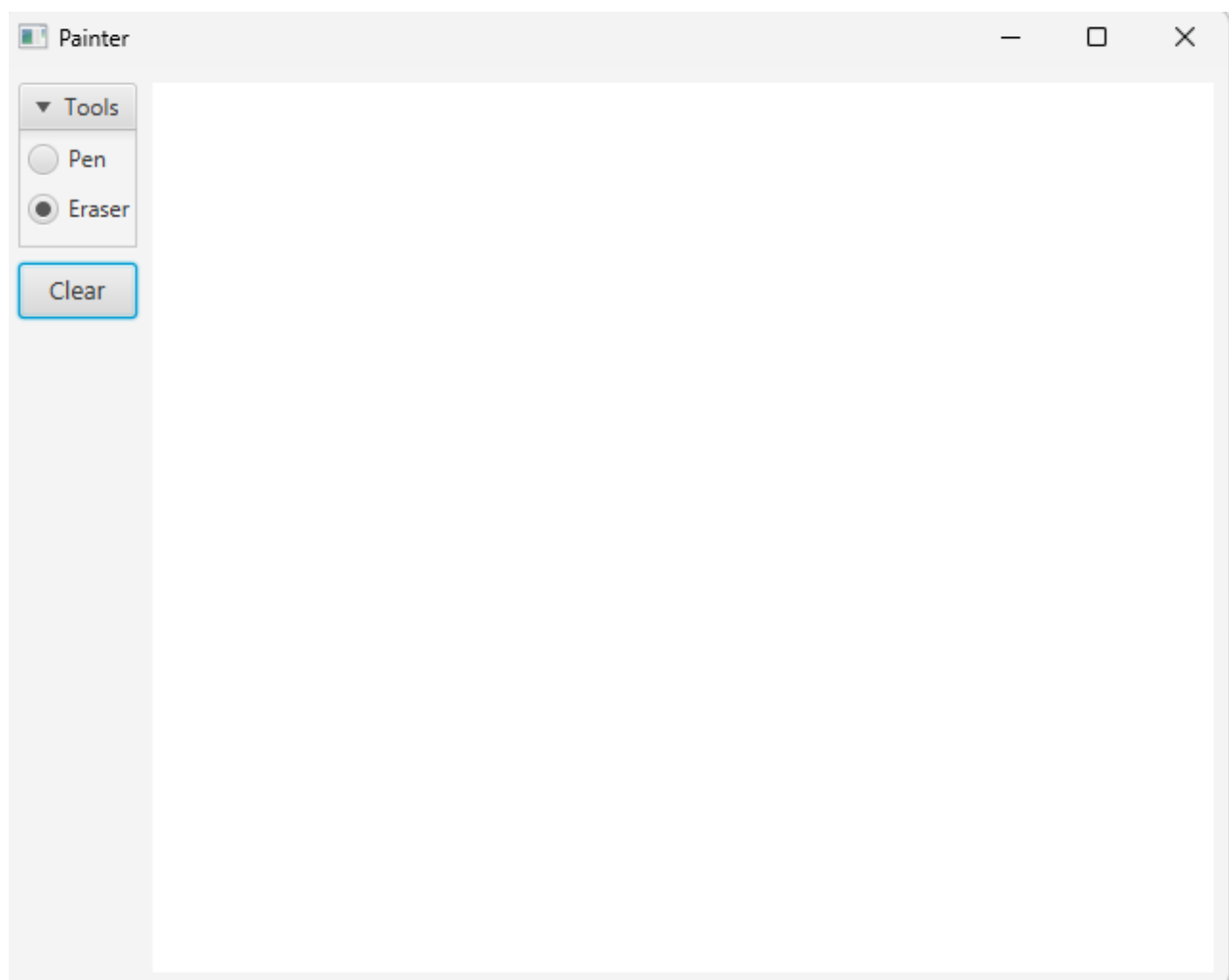
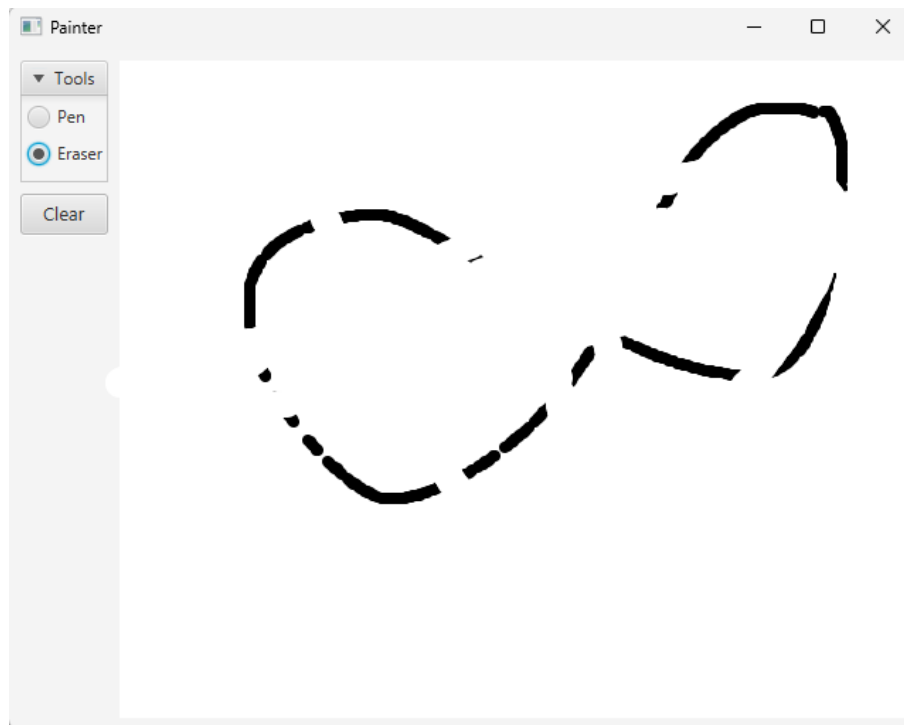
PainterController.java

```
1 package hust.soict.dsai.javafx;
2
3 import javafx.event.ActionEvent;
4 import javafx.fxml.FXML;
5 import javafx.scene.control.RadioButton;
6 import javafx.scene.input.MouseEvent;
7 import javafx.scene.layout.Pane;
8 import javafx.scene.paint.Color;
9 import javafx.scene.shape.Circle;
10
11 public class PainterController {
12
13     @FXML
14     private Pane drawingAreaPane;
15
16     @FXML
17     private RadioButton eraser; // ID của nút tẩy
18
19     @FXML
20     private RadioButton pen; // ID của nút bút
21
22     @FXML
23     void clearButtonPressed(ActionEvent event) {
24         drawingAreaPane.getChildren().clear();
25     }
26
27     @FXML
28     void drawingAreaMouseDragged(MouseEvent event) {
29         if (event.getX() < 0 || event.getY() < 0 ||
30             event.getX() > drawingAreaPane.getWidth() ||
31             event.getY() > drawingAreaPane.getHeight()) {
32             return;
33         }
34         Color inkColor = Color.BLACK;
35         if (eraser.isSelected()) {
36             inkColor = Color.WHITE;
37         }
38
39         double size = eraser.isSelected() ? 10 : 4;
40
41         Circle newCircle = new Circle(event.getX(), event.getY(), size, inkColor);
42         drawingAreaPane.getChildren().add(newCircle);
43     }
44 }
```

Painter.fxml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?import javafx.scene.text.*?>
3 <?import javafx.scene.control.*?>
4 <?import javafx.geometry.*?>
5 <?import java.lang.*?>
6 <?import javafx.scene.layout.*?>
7 <?import javafx.scene.layout.BorderPane?>
8
9 <BorderPane xmlns="http://javafx.com/javafx/8" xmlns:fx="http://javafx.com/fxml/1" fx:controller="hust.soict.dsai.javafx.PainterController">
10 <center>
11 <BorderPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="480.0" prefWidth="640.0" BorderPane.alignment="CENTER">
12 <padding>
13 <Insets bottom="8.0" left="8.0" right="8.0" top="8.0" />
14 </padding>
15
16 <left>
17 <VBox maxHeight="1.7976931348623157E308" spacing="8.0" BorderPane.alignment="CENTER">
18 <BorderPane.margin>
19 <Insets right="8.0" />
20 </BorderPane.margin>
21 <children>
22 <TitlePane animated="false" text="Tools">
23 <content>
24 <AnchorPane minHeight="0.0" minWidth="0.0" prefHeight="60.0" prefWidth="60.0">
25 <children>
26 <RadioButton fx:id="pen" layoutX="4.0" layoutY="6.0" mnemonicParsing="false" selected="true" text="Pen">
27 <toggleGroup>
28 <ToggleGroup fx:id="toolsGroup" />
29 </toggleGroup>
30 </RadioButton>
31 <RadioButton fx:id="eraser" layoutX="4.0" layoutY="32.0" mnemonicParsing="false" text="Eraser" toggleGroup="$toolsGroup" />
32 </children>
33 </AnchorPane>
34 </content>
35 </TitlePane>
36
37 <Button maxWidth="1.7976931348623157E308" mnemonicParsing="false" onAction="#clearButtonPressed" text="Clear">
38 <font>
39 <Font size="13.0" />
40 </font>
41 </Button>
42 </children>
43 </VBox>
44 </left>
45
46 <center>
47 <Pane fx:id="drawingAreaPane" onHouseDragged="#drawingAreaHouseDragged" prefHeight="200.0" prefWidth="200.0" style="-fx-background-color: white;" BorderPane.alignment="CENTER" />
48 </center>
49
50 </BorderPane>
51 </center>
52 </BorderPane>
53
```





5. Hust.soict.dsai.aims.screen

MediaStore.java

StoreScreen.java

```
1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.cart.Cart;
4 import hust.soict.dsai.aims.media.Media;
5 import hust.soict.dsai.aims.store.Store;
6
7 import javafx.collections.ObservableList;
8
9 import javax.swing.*;
10 import java.awt.*;
11 import java.awt.event.ActionEvent;
12 import java.awt.event.ActionListener;
13
14 public class StoreScreen extends JFrame {
15     public static StoreScreen INSTANCE;
16     private Store store;
17     private Cart cart;
18
19
20     // ===== MENU BAR =====
21     JMenuBar createMenuBar() {
22         JMenu menu = new JMenu("Options");
23         JMenu smUpdateStore = new JMenu("Update Store");
24
25         JMenuItem addBook = new JMenuItem("Add Book");
26         addBook.addActionListener(e -> new AddBookToStoreScreen(store, this));
27         smUpdateStore.add(addBook);
28
29         JMenuItem addCD = new JMenuItem("Add CD");
30         addCD.addActionListener(e -> new AddCompactDiscToStoreScreen(store, this));
31         smUpdateStore.add(addCD);
32
33         JMenuItem addDVD = new JMenuItem("Add DVD");
34         addDVD.addActionListener(e -> new AddDigitalVideoDiscToStoreScreen(store, this));
35         smUpdateStore.add(addDVD);
36
37         JMenuItem viewStore = new JMenuItem("View Store");
38         viewStore.addActionListener(e -> new StoreScreen(store, cart));
39
40         JMenuItem viewCart = new JMenuItem("View Cart");
41         viewCart.addActionListener(e -> {
42             new CartScreen(store, cart);
43         });
44     }
45 }
```

```

48 // ----- Play button -----
49 if (media instanceof Playable) {
50     JButton btnPlay = new JButton("Play");
51     btnPlay.addActionListener(e -> {
52         JDialog dialog = new JDialog();
53         dialog.setTitle("Playing");
54         dialog.setSize(250, 120);
55
56         JLabel infoLabel = new JLabel("Playing: " + media.getTitle(), SwingConstants.CENTER);
57         dialog.add(infoLabel);
58
59         dialog.setLocationRelativeTo(null);
60         dialog.setVisible(true);
61     });
62     container.add(btnPlay);
63 }
64
65 JButton btnDelete = new JButton("Delete");
66 btnDelete.addActionListener(e -> {
67     int confirm = JOptionPane.showConfirmDialog(this,
68         "Do you want to delete '" + media.getTitle() + "' from the Store?",
69         "Confirm Delete", JOptionPane.YES_NO_OPTION);
70
71     if (confirm == JOptionPane.YES_OPTION) {
72         store.removeMedia(media);
73
74         if (StoreScreen.INSTANCE != null) {
75             StoreScreen.INSTANCE.refresh();
76         }
77
78         JOptionPane.showMessageDialog(this, "Deleted successfully!");
79     }
80 });
81 container.add(btnDelete);
82 this.add(Box.createVerticalGlue());
83 this.add(title);
84 this.add(cost);
85 this.add(Box.createVerticalGlue());
86 this.add(container);
87
88 this.setBorder(BorderFactory.createLineBorder(Color.BLACK));
89 }
90 }

```

```

46     menu.add(smUpdateStore);
47     menu.add(viewStore);
48     menu.add(viewCart);
49
50     JMenuBar menuBar = new JMenuBar();
51     menuBar.setLayout(new FlowLayout(FlowLayout.LEFT));
52     menuBar.add(menu);
53
54     return menuBar;
55 }
56
57
58 // ===== HEADER =====
59 JPanel createHeader() {
60     JPanel header = new JPanel();
61     header.setLayout(new BoxLayout(header, BoxLayout.X_AXIS));
62
63     JLabel title = new JLabel("AIMS");
64     title.setFont(new Font(title.getFont().getName(), Font.PLAIN, 50));
65     title.setForeground(Color.CYAN);
66
67     JButton cartBtn = new JButton("View cart");
68     cartBtn.setPreferredSize(new Dimension(100, 50));
69     cartBtn.setMaximumSize(new Dimension(100, 50));
70
71     cartBtn.addActionListener(new ActionListener() {
72         @Override
73         public void actionPerformed(ActionEvent e) {
74             new CartScreen(store, cart);
75         }
76     });
77
78     header.add(Box.createRigidArea(new Dimension(10, 10)));
79     header.add(title);
80     header.add(Box.createHorizontalGlue());
81     header.add(cartBtn);
82     header.add(Box.createRigidArea(new Dimension(10, 10)));
83
84     return header;
85 }
86
87 // ===== CENTER (GRID 3x3) =====
88 JPanel createCenter() {
89     JPanel center = new JPanel();
90     center.setLayout(new GridLayout(3, 3, 2, 2));
91
92     ObservableList<Media> mediaInStore = store.getItemsInStore();
93
94     // Always create 9 cells
95     for (int i = 0; i < 9; i++) {
96         if (i < mediaInStore.size()) {
97             center.add(new MediaStore(mediaInStore.get(i), cart));
98         } else {
99             center.add(new JPanel()); // empty cell
100         }
101     }
102
103     return center;
104 }
105
106 // ===== REFRESH =====
107 public void refresh() {
108     this.getContentPane().removeAll();
109     this.getContentPane().add(createNorth(), BorderLayout.NORTH);
110     this.getContentPane().add(createCenter(), BorderLayout.CENTER);
111     this.revalidate();
112     this.repaint();
113 }
114
115 JPanel createNorth() {
116     JPanel north = new JPanel();
117     north.setLayout(new BoxLayout(north, BoxLayout.Y_AXIS));
118     north.add(createMenuBar());
119     north.add(createHeader());
120     return north;
121 }

```

```

123 // ===== CONSTRUCTOR =====
124 public StoreScreen(Store store, Cart cart) {
125     INSTANCE = this;
126     this.store = store;
127     this.cart = cart;
128
129     Container cp = getContentPane();
130     cp.setLayout(new BorderLayout());
131
132     cp.add(createNorth(), BorderLayout.NORTH);
133     cp.add(createCenter(), BorderLayout.CENTER);
134
135     this.setTitle("Store");
136     this.setSize(1024, 768);
137     this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
138     this.setLocationRelativeTo(null);
139     this.setVisible(true);
140 }
141 }

```

CartScreen.java

```

1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.cart.Cart;
4 import hust.soict.dsai.aims.store.Store;
5 import javafx.application.Platform;
6 import javafx.embed.swing.JFXPanel;
7 import javafx.fxml.FXMLLoader;
8 import javafx.scene.Parent;
9 import javafx.scene.Scene;
10
11 import javax.swing.*;
12 import java.awt.event.ActionEvent;
13 import java.awt.event.ActionListener;
14 import java.io.IOException;
15
16 public class CartScreen extends JFrame {
17
18     private Cart cart;
19     private Store store;
20
21     public CartScreen(Store store, Cart cart) {
22         super();
23
24         Platform.setImplicitExit(false);
25
26         this.store = store;
27         this.cart = cart;
28
29         JFXPanel fxPanel = new JFXPanel();
30         this.add(fxPanel);
31
32         this.setTitle("Cart");
33         this.setSize(1024, 1024);
34         this.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); // Đóng Cart thì chỉ tắt Cart
35         this.setLocationRelativeTo(null);
36
37         this.setJMenuBar(createMenuBar());
38
39         this.setVisible(true);

```

```

41Ⓣ Platform.runLater(new Runnable() {
42Ⓣ     @Override
43Ⓣ     public void run() {
44Ⓣ         try {
45Ⓣ             FXMLLoader loader = new FXMLLoader(getClass().getResource("/hust/soict/dsai/aims/screen/cart.fxml"));
46Ⓣ
47Ⓣ             CartScreenController controller = new CartScreenController(cart);
48Ⓣ             loader.setController(controller);
49Ⓣ
50Ⓣ             Parent root = loader.load();
51Ⓣ             fxPanel.setScene(new Scene(root));
52Ⓣ         } catch (IOException e) {
53Ⓣ             e.printStackTrace();
54Ⓣ         }
55Ⓣ     }
56Ⓣ });
57Ⓣ }
58Ⓣ
59Ⓣ private JMenuBar createMenuBar() {
60Ⓣ     JMenuBar menuBar = new JMenuBar();
61Ⓣ     JMenu menuOptions = new JMenu("Options");
62Ⓣ
63Ⓣ     JMenuItem viewStore = new JMenuItem("View Store");
64Ⓣ     viewStore.addActionListener(new ActionListener() {
65Ⓣ         @Override
66Ⓣ         public void actionPerformed(ActionEvent e) {
67Ⓣ             if (StoreScreen.INSTANCE != null) {
68Ⓣ                 StoreScreen.INSTANCE.setVisible(true);
69Ⓣ                 StoreScreen.INSTANCE.refresh();
70Ⓣ             } else {
71Ⓣ                 new StoreScreen(store, cart);
72Ⓣ             }
73Ⓣ             dispose(); // Đóng màn hình Cart hiện tại
74Ⓣ         }
75Ⓣ     });
76Ⓣ
77Ⓣ     menuOptions.add(viewStore);
78Ⓣ     menuBar.add(menuOptions);
79Ⓣ     return menuBar;
80Ⓣ }
81Ⓣ }

```

CartScreenController.java

```
1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.cart.Cart;
4 import hust.soict.dsai.aims.exception.PlayerException;
5 import hust.soict.dsai.aims.media.CompactDisc;
6 import hust.soict.dsai.aims.media.DigitalVideoDisc;
7 import hust.soict.dsai.aims.media.Media;
8 import hust.soict.dsai.aims.media.Playable;
9 import javafx.application.Platform;
10 import javafx.beans.value.ChangeListener;
11 import javafx.beans.value.ObservableValue;
12 import javafx.collections.ListChangeListener;
13 import javafx.event.ActionEvent;
14 import javafx.fxml.FXML;
15 import javafx.scene.control.*;
16 import javafx.scene.control.cell.PropertyValueFactory;
17 import javafx.scene.control.Alert.AlertType;
18
19 public class CartScreenController {
20
21     private Cart cart;
22
23     @FXML private TableView<Media> tblMedia;
24     @FXML private TableColumn<Media, String> colMediaTitle;
25     @FXML private TableColumn<Media, String> colMediaCategory;
26     @FXML private TableColumn<Media, Float> colMediaCost;
27     @FXML private Button btnPlay;
28     @FXML private Button btnRemove;
29     @FXML private TextField tfFilter;
30     @FXML private RadioButton radioBtnFilterId;
31     @FXML private RadioButton radioBtnFilterTitle;
32     @FXML private Label lblTotalCost;
33     @FXML private Button btnPlaceOrder;
34
35     private ToggleGroup filterGroup = new ToggleGroup();
36
37     public CartScreenController(Cart cart) {
38         super();
39         this.cart = cart;
40     }
41 }
```

```

42 @FXML
43 private void initialize() {
44     // cell factories
45     colMediaTitle.setCellValueFactory(new PropertyValueFactory<Media, String>("title"));
46     colMediaCategory.setCellValueFactory(new PropertyValueFactory<Media, String>("category"));
47     colMediaCost.setCellValueFactory(new PropertyValueFactory<Media, Float>("cost"));
48     tblMedia.setItems(cart.getItemsOrdered());
49
50     // hide buttons initially
51     btnPlay.setVisible(false);
52     btnRemove.setVisible(false);
53
54     radioBtnFilterId.setToggleGroup(filterGroup);
55     radioBtnFilterTitle.setToggleGroup(filterGroup);
56     radioBtnFilterId.setSelected(true); // default
57
58     tblMedia.getSelectionModel().selectedItemProperty().addListener(
59         new ChangeListener<Media>() {
60
61             @Override
62             public void changed(ObservableValue<? extends Media> observable, Media oldValue,
63                 Media newValue) {
64                 System.out.println("DEBUG: Đã chọn media: " + (newValue != null ? newValue.getTitle() : "null"));
65                 if (newValue != null) {
66                     updateButtonBar(newValue);
67                 }
68             }
69         });
70
71     tfFilter.textProperty().addListener(new ChangeListener<String>() {
72
73         @Override
74         public void changed(ObservableValue<? extends String> observable, String oldValue,
75             String newValue) {
76             if (newValue != null) {
77                 showFilteredMedia(newValue);
78             }
79         }
80     });
81
82     cart.addListChangeListener((ListChangeListener.Change<? extends Media> change) -> {
83         Platform.runLater(() -> {
84             updateTotalCost();
85             // if no item selected, hide buttons
86             if (cart.getItemsOrdered().isEmpty()) {
87                 btnPlay.setVisible(false);
88                 btnRemove.setVisible(false);
89             }
90         });
91     });
92     updateTotalCost();
93 }

```

```

95 // update play/remove visibility depending on selected media
96 void updateButtonBar(Media media) {
97     btnRemove.setVisible(true);
98     btnPlay.setVisible(media instanceof Playable);
99 }
100
101 @FXML
102 void btnRemovePressed(ActionEvent event) {
103     try {
104         Media media = tblMedia.getSelectionModel().getSelectedItem();
105
106         if (media != null) {
107             cart.removeMedia(media);
108
109             updateTotalCost();
110
111             btnRemove.setVisible(false);
112             btnPlay.setVisible(false);
113         } else {
114             System.out.println("Chưa chọn sản phẩm nào để xóa!");
115         }
116     } catch (Exception e) {
117         e.printStackTrace();
118         Alert alert = new Alert(Alert.AlertType.ERROR);
119         alert.setTitle("Lỗi khi xóa");
120         alert.setContentText(e.getMessage());
121         alert.showAndWait();
122     }
123 }
124
125 @FXML
126 void btnPlayPressed(ActionEvent event) {
127     Media media = tblMedia.getSelectionModel().getSelectedItem();
128     if (media instanceof Playable) {
129         try {
130             ((Playable) media).play();
131
132             Alert alert = new Alert(Alert.AlertType.INFORMATION);
133             alert.setTitle("Playing");
134             alert.setHeaderText("Playing Media");
135             alert.setContentText("Media is playing... Check console for details.");
136             alert.showAndWait();
137
138         } catch (PlayerException e) {
139             Alert alert = new Alert(Alert.AlertType.ERROR);
140             alert.setTitle("Illegal DVD Length");
141             alert.setHeaderText("ERROR: Media cannot be played");
142             alert.setContentText(e.getMessage()); // Lấy nội dung lỗi từ PlayerException
143             alert.showAndWait();
144         }
145     }
146 }

```

```

148 @FXML
149 void btnPlaceOrderPressed(ActionEvent event) {
150     if (cart.getItemsOrdered().isEmpty()) {
151         Alert alert = new Alert(AlertType.INFORMATION, "Cart is empty.");
152         alert.showAndWait();
153         return;
154     }
155
156     Alert confirm = new Alert(AlertType.CONFIRMATION, "Place order? This will clear the cart.", ButtonType.YES, ButtonType.NO);
157     confirm.setHeaderText(null);
158     confirm.showAndWait().ifPresent(response -> {
159         if (response == ButtonType.YES) {
160             cart.clear();
161             updateTotalCost();
162             Alert info = new Alert(AlertType.INFORMATION, "Order placed successfully!");
163             info.setHeaderText(null);
164             info.showAndWait();
165         }
166     });
167 }
168
169 // filter implementation
170 private void showFilteredMedia(String filter) {
171     if (filter == null || filter.isEmpty()) {
172         tblMedia.setItems(cart.getItemsOrdered());
173         return;
174     }
175
176     String keyword = filter.toLowerCase();
177
178     if (radioBtnFilterId.isSelected()) {
179         tblMedia.setItems(
180             cart.getItemsOrdered().filtered(
181                 item -> Integer.toString(item.getId()).contains(keyword)
182             )
183         );
184     } else {
185         tblMedia.setItems(
186             cart.getItemsOrdered().filtered(
187                 item -> item.getTitle().toLowerCase().contains(keyword)
188             )
189         );
190     }
191 }
192
193 // update total label
194 private void updateTotalCost() {
195     float total = cart.totalCost();
196     lblTotalCost.setText(String.format("%.2f $", total));
197 }
198 }
---
```

AddItemToStoreScreen.java

```
1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.store.Store;
4 import javax.swing.*;
5 import java.awt.*;
6
7 public abstract class AddItemToStoreScreen extends JFrame {
8     protected Store store;
9     protected StoreScreen storeScreen; // <--- Thêm biến này
10
11     // Cập nhật Constructor nhận thêm StoreScreen
12 public AddItemToStoreScreen(Store store, StoreScreen storeScreen) {
13     this.store = store;
14     this.storeScreen = storeScreen; // <--- Lưu lại
15
16     this.setLayout(new BorderLayout());
17     this.add(createMenuBar(), BorderLayout.NORTH);
18     this.setTitle("Add Item");
19     this.setSize(400, 300);
20     this.setVisible(true);
21 }
22
23 protected JMenuBar createMenuBar() {
24     JMenu menu = new JMenu("Options");
25
26     JMenu smUpdateStore = new JMenu("Update Store");
27     smUpdateStore.add(new JMenuItem("Add Book"));
28     smUpdateStore.add(new JMenuItem("Add CD"));
29     smUpdateStore.add(new JMenuItem("Add DVD"));
30
31     menu.add(smUpdateStore);
32     menu.add(new JMenuItem("View store"));
33
34     JMenuBar bar = new JMenuBar();
35     bar.add(menu);
36     return bar;
37 }
38 }
```

AddBookToStoreScreen.java

```
1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.media.Book;
4 import hust.soict.dsai.aims.store.Store;
5
6 import javax.swing.*;
7 import java.awt.*;
8 import java.util.ArrayList;
9
10 public class AddBookToStoreScreen extends AddItemToStoreScreen {
11
12     public AddBookToStoreScreen(Store store, StoreScreen storeScreen) {
13         super(store, storeScreen);
14
15         JPanel form = new JPanel(new GridLayout(6, 2, 5, 5));
16
17         JTextField tfTitle = new JTextField();
18         JTextField tfCategory = new JTextField();
19         JTextField tfCost = new JTextField();
20         JTextField tfAuthorName = new JTextField();
21
22         // Danh sách tác giả
23         DefaultListModel<String> authorsModel = new DefaultListModel<>();
24         JList<String> authorsList = new JList<>(authorsModel);
25         JScrollPane authorScroll = new JScrollPane(authorsList);
26
27         // Nút thêm tác giả
28         JButton btnAddAuthor = new JButton("Add Author");
29         btnAddAuthor.addActionListener(e -> {
30             String author = tfAuthorName.getText().trim();
31             if (!author.isEmpty()) {
32                 authorsModel.addElement(author);
33                 tfAuthorName.setText("");
34             }
35         });
36
37         // Form input
38         form.add(new JLabel("Title:"));
39         form.add(tfTitle);
40
41         form.add(new JLabel("Category:"));
42         form.add(tfCategory);
43
44         form.add(new JLabel("Cost:"));
45         form.add(tfCost);
46
47         form.add(new JLabel("Author name:"));
48         form.add(tfAuthorName);
49
50         form.add(new JLabel(" "));
51         form.add(btnAddAuthor);
52     }
53 }
```

```

53 form.add(new JLabel("Authors list:"));
54 form.add(authorScroll);
55
56 // Nút Add Book
57 JButton btnAdd = new JButton("Add Book");
58 btnAdd.addActionListener(e -> {
59     try {
60         String title = tfTitle.getText();
61         String category = tfCategory.getText();
62         float cost = Float.parseFloat(tfCost.getText());
63         if (cost < 0) {
64             throw new IllegalArgumentException("Cost cannot be negative!");
65         }
66         ArrayList<String> authors = new ArrayList<>();
67         for (int i = 0; i < authorsModel.size(); i++) {
68             authors.add(authorsModel.get(i));
69         }
70
71         Book book = new Book(title, category, authors, cost);
72         store.addMedia(book);
73
74         storeScreen.refresh();
75
76         JOptionPane.showMessageDialog(this, "Book added!");
77
78     } catch (NumberFormatException nfe) {
79         JOptionPane.showMessageDialog(this, "Error: Cost must be a valid number!", "Input Error", JOptionPane.ERROR_MESSAGE);
80     } catch (IllegalArgumentException iae) {
81         JOptionPane.showMessageDialog(this, "Error: " + iae.getMessage(), "Input Error", JOptionPane.ERROR_MESSAGE);
82     }
83 });
84
85 // Gắn vào window
86 this.add(form, BorderLayout.CENTER);
87 this.add(btnAdd, BorderLayout.SOUTH);
88
89 this.setTitle("Add Book");
90 }
91 }

```

AddDigitalVideoDiscToStore.java

```
1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.media.DigitalVideoDisc;
4
5
6
7
8
9 public class AddDigitalVideoDiscToStoreScreen extends AddItemToStoreScreen {
10
11     public AddDigitalVideoDiscToStoreScreen(Store store, StoreScreen storeScreen) {
12         super(store, storeScreen);
13
14         JPanel form = new JPanel(new GridLayout(5, 2));
15
16         JTextField tfTitle = new JTextField();
17         JTextField tfCategory = new JTextField();
18         JTextField tfCost = new JTextField();
19         JTextField tfLength = new JTextField();
20         JTextField tfDirector = new JTextField();
21
22         form.add(new JLabel("Title:"));
23         form.add(tfTitle);
24
25         form.add(new JLabel("Category:"));
26         form.add(tfCategory);
27
28         form.add(new JLabel("Cost:"));
29         form.add(tfCost);
30
31         form.add(new JLabel("Length:"));
32         form.add(tfLength);
33
34         form.add(new JLabel("Director:"));
35         form.add(tfDirector);
36
37         JButton btnAdd = new JButton("Add");
38         btnAdd.addActionListener(e -> {
39             try {
40                 String title = tfTitle.getText();
41                 String category = tfCategory.getText();
42                 String director = tfDirector.getText();
43
44                 if (tfCost.getText().isEmpty()) throw new IllegalArgumentException("Cost cannot be empty!");
45                 float cost = Float.parseFloat(tfCost.getText());
46                 if (cost < 0) {
47                     throw new IllegalArgumentException("Cost cannot be negative!");
48                 }
49
50                 if (tfLength.getText().isEmpty()) throw new IllegalArgumentException("Length cannot be empty!");
51                 int length = Integer.parseInt(tfLength.getText());
52                 if (length <= 0) {
53                     throw new IllegalArgumentException("Length must be positive!");
54                 }
55             }
56         });
57     }
58 }
```

AddCompactDiscToStore.java

```
1 package hust.soict.dsai.aims.screen;
2
3 import hust.soict.dsai.aims.media.CompactDisc;
4 import hust.soict.dsai.aims.media.Track;
5 import hust.soict.dsai.aims.store.Store;
6
7 import javax.swing.*;
8 import java.awt.*;
9 import java.util.ArrayList;
10
11 public class AddCompactDiscToStoreScreen extends AddItemToStoreScreen {
12
13     public AddCompactDiscToStoreScreen(Store store, StoreScreen storeScreen) {
14         super(store, storeScreen);
15
16         JPanel form = new JPanel(new GridLayout(7, 2, 5, 5));
17
18         JTextField tfTitle = new JTextField();
19         JTextField tfCategory = new JTextField();
20         JTextField tfDirector = new JTextField();
21         JTextField tfArtist = new JTextField();
22         JTextField tfCost = new JTextField();
23
24         JTextField tfTrackTitle = new JTextField();
25         JTextField tfTrackLength = new JTextField();
26
27         DefaultListModel<Track> trackModel = new DefaultListModel<>();
28         JList<Track> trackList = new JList<>(trackModel);
29         JScrollPane trackScroll = new JScrollPane(trackList);
30
31         JButton btnAddTrack = new JButton("Add Track");
32         btnAddTrack.addActionListener(e -> {
33             try {
34                 String tTitle = tfTrackTitle.getText().trim();
35                 int tLength = Integer.parseInt(tfTrackLength.getText().trim());
36
37                 if (!tTitle.isEmpty() && tLength > 0) {
38                     trackModel.addElement(new Track(tTitle, tLength));
39                     tfTrackTitle.setText("");
40                     tfTrackLength.setText("");
41                 }
42             } catch (Exception ex) {
43                 JOptionPane.showMessageDialog(this, "Invalid track length!");
44             }
45         });
46
47         form.add(new JLabel("Title:"));
48         form.add(tfTitle);
49
50         form.add(new JLabel("Category:"));
51         form.add(tfCategory);
52
53     });
54
55     this.add(form, BorderLayout.CENTER);
56     this.add(btnAdd, BorderLayout.SOUTH);
57 }
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77 }
```

```

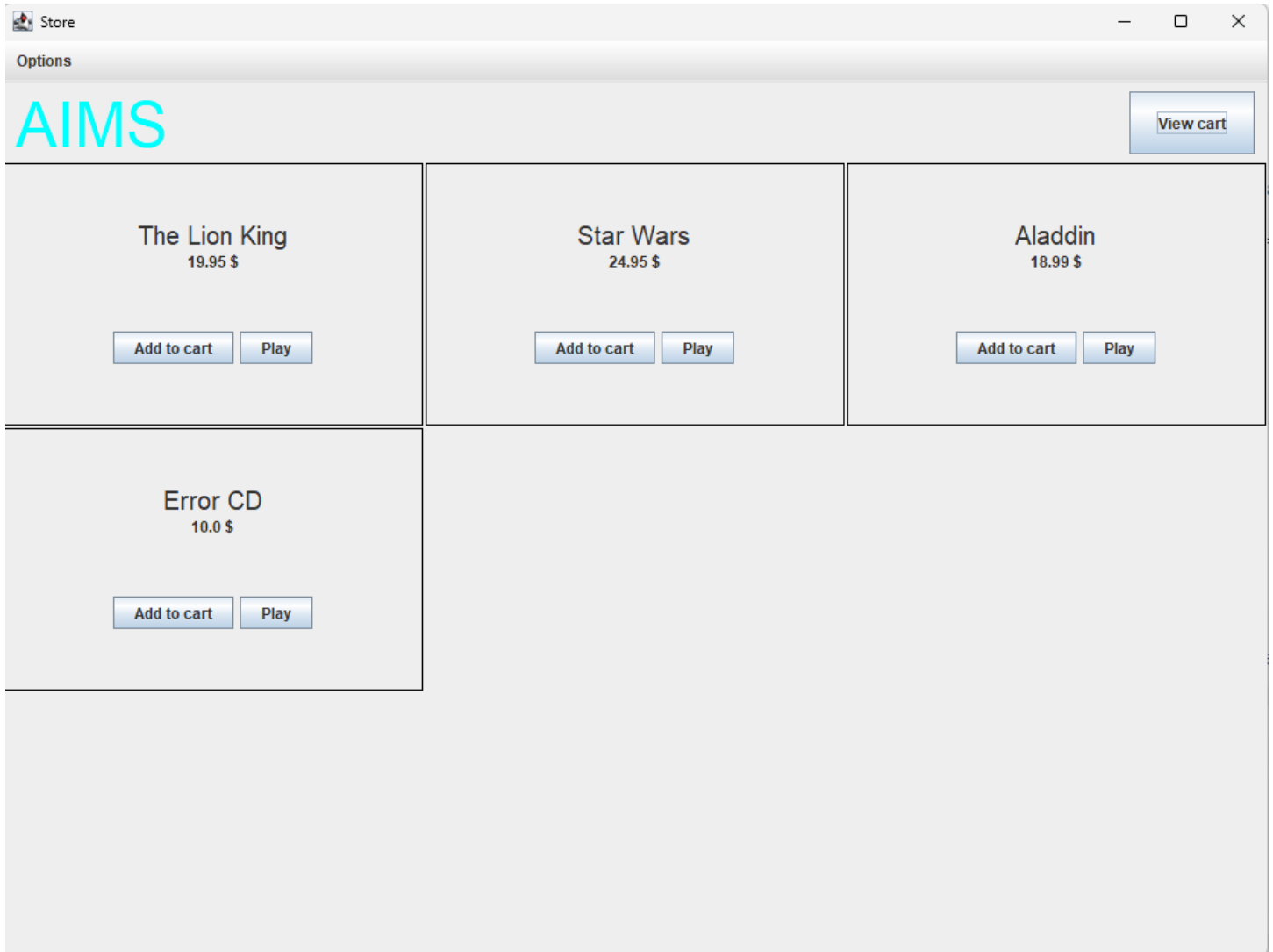
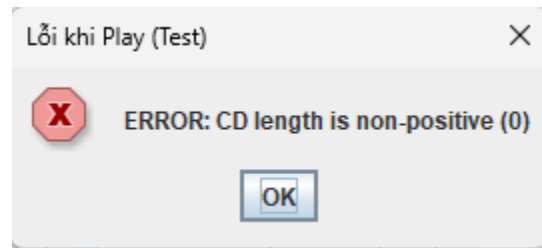
53 form.add(new JLabel("Director:"));
54 form.add(tfDirector);
55
56 form.add(new JLabel("Artist:"));
57 form.add(tfArtist);
58
59 form.add(new JLabel("Cost:"));
60 form.add(tfCost);
61
62 form.add(new JLabel("Track Title:"));
63 form.add(tfTrackTitle);
64
65 form.add(new JLabel("Track Length:"));
66 form.add(tfTrackLength);
67
68 form.add(new JLabel(""));
69 form.add(btnAddTrack);
70
71 form.add(new JLabel("Tracks:"));
72 form.add(trackScroll);
73
74 JButton btnAdd = new JButton("Add CD");
75 btnAdd.addActionListener(e -> {
76     try {
77         String title = tfTitle.getText();
78         String category = tfCategory.getText();
79         String director = tfDirector.getText();
80         String artist = tfArtist.getText();
81         float cost = Float.parseFloat(tfCost.getText());
82         if (cost < 0) {
83             throw new IllegalArgumentException("Cost cannot be negative!");
84         }
85
86         ArrayList<Track> tracks = new ArrayList<>();
87         int totalLength = 0;
88
89         for (int i = 0; i < trackModel.size(); i++) {
90             Track t = trackModel.get(i);
91             if (t.getLength() < 0) {
92                 throw new IllegalArgumentException("Length cannot be negative!");
93             }
94             tracks.add(t);
95             totalLength += t.getLength();
96         }
97
98         CompactDisc cd = new CompactDisc(
99             title,
100             category,
101             director,
102             artist,
103             tracks,
104             totalLength,
105             cost
106         );
107
108         store.addMedia(cd);
109         storeScreen.refresh();
110
111         JOptionPane.showMessageDialog(this, "CD added!");
112     } catch (Exception ex) {
113         JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage());
114     }
115 });
116
117 this.add(form, BorderLayout.CENTER);
118 this.add(btnAdd, BorderLayout.SOUTH);
119 this.setTitle("Add Compact Disc");
120 }

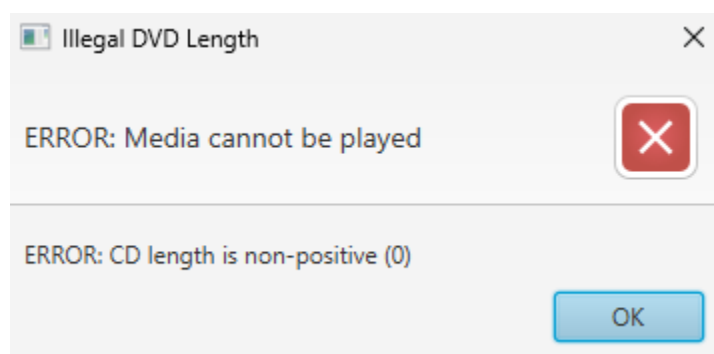
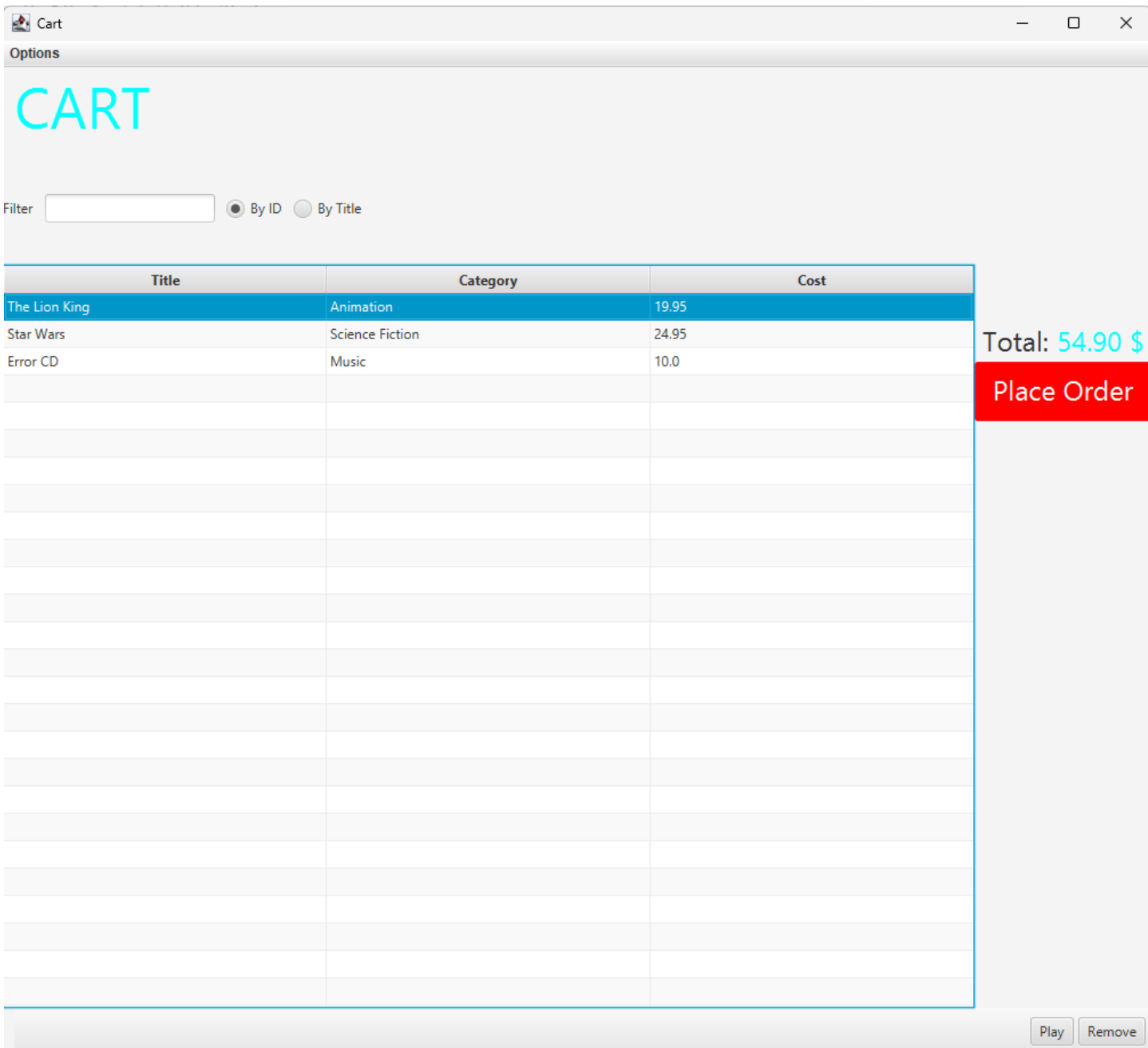
```

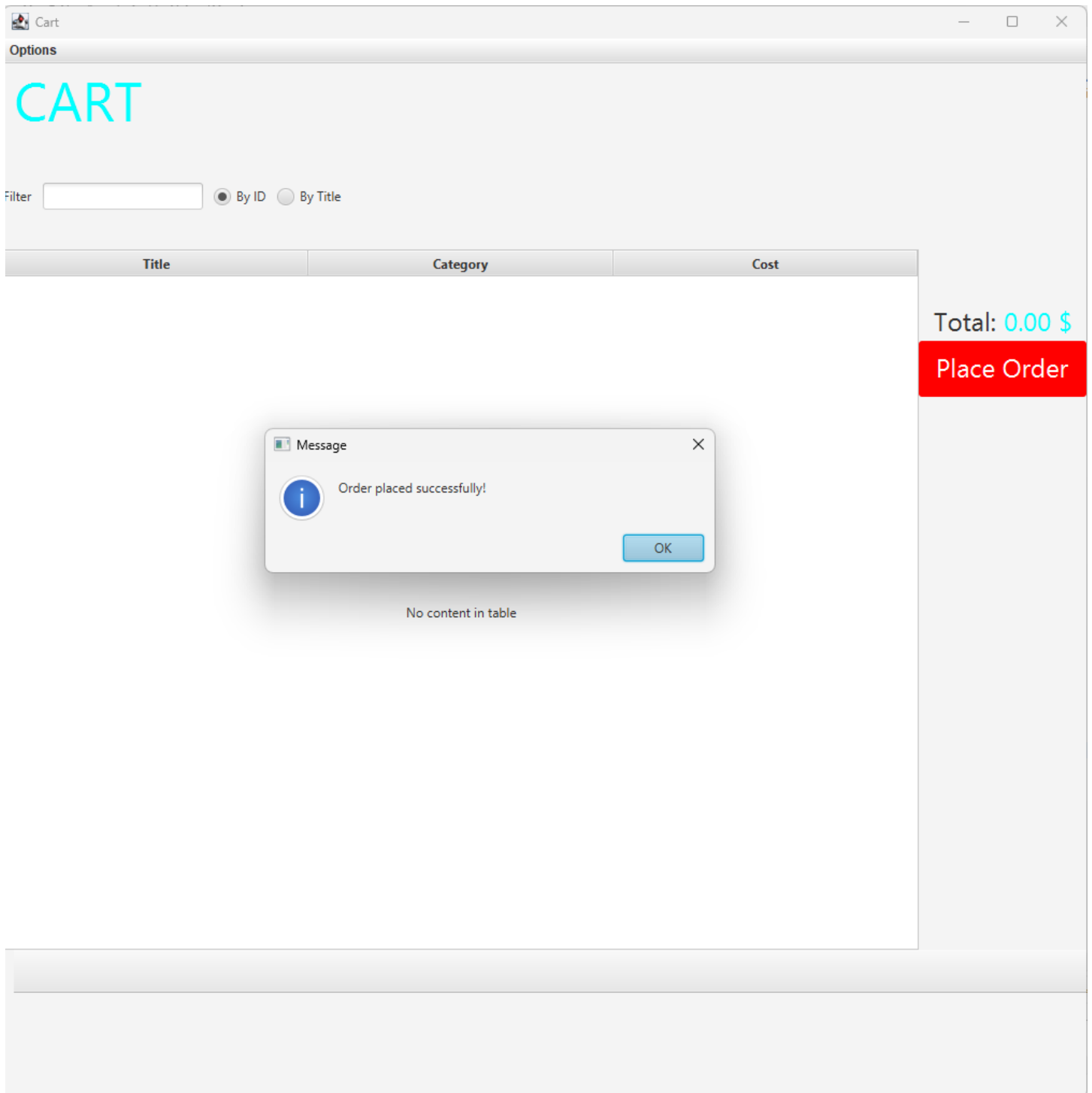
Aims.java

```
1 package hust.soict.dsai.aims;
2
3 import hust.soict.dsai.aims.cart.Cart;
4 import hust.soict.dsai.aims.exception.PlayerException; // Import Exception
5 import hust.soict.dsai.aims.media.DigitalVideoDisc;
6 import hust.soict.dsai.aims.media.Book;
7 import hust.soict.dsai.aims.media.CompactDisc;
8 import hust.soict.dsai.aims.media.Track;
9 import hust.soict.dsai.aims.screen.StoreScreen;
10 import hust.soict.dsai.aims.store.Store;
11
12 import javax.swing.JOptionPane; // Dùng để hiện dialog theo yêu cầu
13 import javax.swing.SwingUtilities;
14 import java.util.ArrayList;
15
16 public class Aims {
17     public static void main(String[] args) {
18         // 1. Khởi tạo Store và Cart
19         Store store = new Store();
20         Cart cart = new Cart();
21
22         // 2. Tạo dữ liệu mẫu
23         DigitalVideoDisc dvd1 = new DigitalVideoDisc("The Lion King", "Animation", "Roger Allers", 87, 19.95f);
24         store.addMedia(dvd1);
25
26         DigitalVideoDisc dvd2 = new DigitalVideoDisc("Star Wars", "Science Fiction", "George Lucas", 87, 24.95f);
27         store.addMedia(dvd2);
28
29         DigitalVideoDisc dvd3 = new DigitalVideoDisc("Aladdin", "Animation", 18.99f);
30         store.addMedia(dvd3);
31
32         ArrayList<Track> tracks = new ArrayList<Track>();
33         CompactDisc cdError = new CompactDisc("Error CD", "Music", "Nobody", tracks, 10.0f);
34         cdError.addTrack(new Track("Bad Track", 0));
35         store.addMedia(cdError);
36
37         System.out.println("---- BẮT ĐẦU TEST EXCEPTION (CONSOLE) ----");
38         try {
39             System.out.println("Đang thử play CD bị lỗi...");
40             cdError.play();
41         } catch (PlayerException e) {
42             System.out.println("Đã bắt được lỗi: " + e.getMessage());
43
44             System.out.println("toString(): " + e.toString());
45
46             e.printStackTrace();
47
48             JOptionPane.showMessageDialog(null, e.getMessage(), "Lỗi khi Play (Test)", JOptionPane.ERROR_MESSAGE);
49         }
50
51         System.out.println("---- KẾT THÚC TEST EXCEPTION ----");
52         try {
53             SwingUtilities.invokeLater(new Runnable() {
54                 @Override
55                 public void run() {
56                     new StoreScreen(store, cart);
57                 }
58             });
59         } catch (Exception e) {
60             e.printStackTrace();
61         }
62     }
63 }
```

- Result:







Besides, we can add or remove more medias from store, sort products in cart and view store from cart to order more products